

Hotel Booking & Management System

A Database-Driven Approach with a Dynamic Pricing Agent

EBU5503 Database Systems — Group Coursework Report (2025/26)

Scenario 2 | May 2026 | Group of 7

1. Detailed Requirements and Assumptions

1.1 Basic Requirements (Scenario 2)

The system records information about **guests, rooms, bookings/reservations, and payments**, as specified by the brief. Each booking belongs to one guest, can contain one or more rooms, and is settled by one or more payment transactions.

1.2 Additional Requirements (Innovation)

Beyond the basics we introduce five realistic extensions, each grounded in real hotel-industry operations:

Feature	Justification & Real-world Mapping
Loyalty Program	Returning guests are rewarded with tiered benefits (Bronze / Silver / Gold / Platinum) – a retention engine standard across Marriott Bonvoy and Hilton Honors.
Dynamic Pricing Rule	Event- and season-based rate adjustment (Christmas, Spring Festival, Summer, weekdays) enables a <i>revenue-management</i> layer on top of static base prices.
Room Type abstraction	Separates physical room from pricing category, eliminating update anomalies (2NF).
Department & Staff	Captures hotel organisation; required for any non-trivial PMS.
Task Log	Tracks cleaning, maintenance and inspection workload per (Staff, Room, Task_Type), enabling KPI analytics.

1.3 Key Assumptions

- A booking can contain multiple rooms across overlapping dates.
- Payments may be partial deposits, full settlements, or refunds linked to cancellations.
- Loyalty membership is optional; a guest is enrolled at most once (1:1).
- Pricing rules are date-windowed and apply globally to all room types; priority breaks overlapping windows.
- Concurrent same-room overlap is prevented by application-layer validation; the database stores a CHECK on date ordering but cannot natively enforce range non-overlap.

2. Conceptual Schema (EER Design)

The schema comprises 11 entities organised into four functional domains: *customer*, *inventory*, *transactional*, and *operational*. The diagram below shows the entity layout, while the textual model that follows names every relationship explicitly.

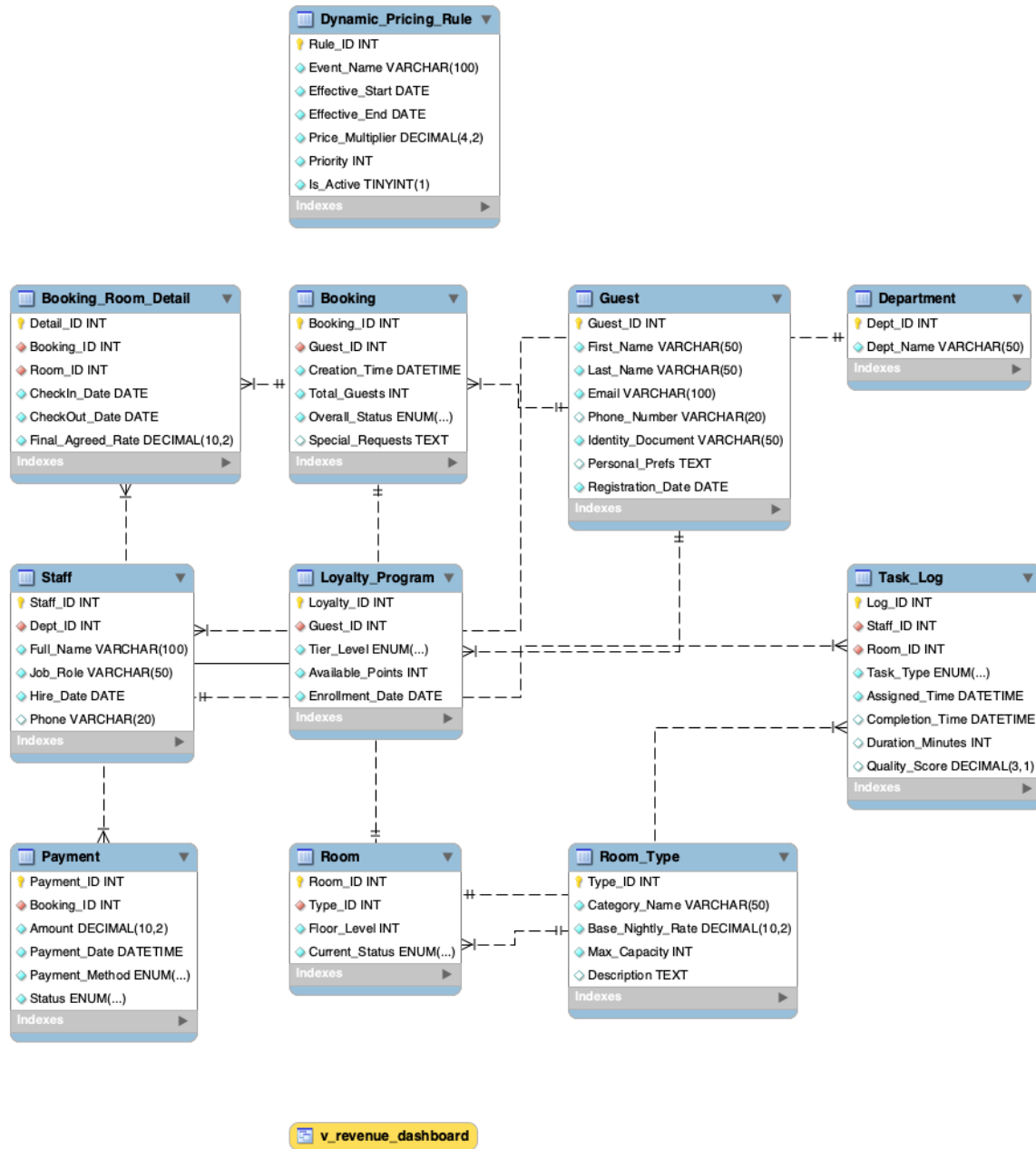


Figure 1: EER diagram of the Hotel Booking & Management System produced via MySQL Workbench Reverse Engineer. Eleven entities span the customer, inventory, transactional and operational domains; BOOKING_ROOM_DETAIL is the associative entity resolving the M:N relationship between BOOKING and ROOM.

Key relationships. (i) BOOKING–ROOM is an M:N relationship, decomposed by the *associative entity* BOOKING_ROOM_DETAIL which carries per-room check-in/out dates and the agreed nightly rate; (ii) STAFF, ROOM, and TASK_TYPE form a *ternary* relationship realised by TASK_LOG; (iii) GUEST–LOYALTY_PROGRAM is 1:1, optional on the guest side; (iv) DYNAMIC_PRICING_RULE is a *global rule entity*, consulted by application logic rather than via a foreign key – this allows rules to evolve independently of the inventory.

3. Relational Schema and Normalisation

3.1 Relational Schema

Relation	Attributes (PK <u>underlined</u> , FK <i>italic</i>)
Guest	<u>Guest_ID</u> , First_Name, Last_Name, Email ^U , Phone, Identity_Doc ^U , Personal_Prefs, Registration_Date
Loyalty_Program	<u>Loyalty_ID</u> , <i>Guest_ID</i> ^U , Tier_Level, Available_Points, Enrollment_Date
Room_Type	<u>Type_ID</u> , Category_Name ^U , Base_Nightly_Rate, Max_Capacity, Description
Room	<u>Room_ID</u> , <i>Type_ID</i> , Floor_Level, Current_Status
Booking	<u>Booking_ID</u> , <i>Guest_ID</i> , Creation_Time, Total_Guests, Overall_Status, Special_Requests
Booking_Room_Detail	<u>Detail_ID</u> , <i>Booking_ID</i> , <i>Room_ID</i> , CheckIn_Date, CheckOut_Date, Final_Agreed_Rate
Payment	<u>Payment_ID</u> , <i>Booking_ID</i> , Amount, Payment_Date, Payment_Method, Status
Dynamic_Pricing_Rule	<u>Rule_ID</u> , Event_Name, Effective_Start, Effective_End, Price_Multiplier, Priority, Is_Active
Department	<u>Dept_ID</u> , Dept_Name ^U
Staff	<u>Staff_ID</u> , <i>Dept_ID</i> , Full_Name, Job_Role, Hire_Date, Phone
Task_Log	<u>Log_ID</u> , <i>Staff_ID</i> , <i>Room_ID</i> , Task_Type, Assigned_Time, Completion_Time, Duration_Minutes, Quality_Score

^U UNIQUE constraint applied.

3.2 Normalisation up to 3NF (BCNF check)

1NF. All attributes are atomic. The draft’s monolithic LOG table conflated booking, room assignment and payment, violating 1NF. It is split into BOOKING, BOOKING_ROOM_DETAIL, PAYMENT. **2NF.** No non-key attribute depends on part of a composite key; the draft’s ROOM(roomNo, price, roomType) is decomposed by abstracting out ROOM_TYPE. **3NF.** No transitive dependencies; PAYMENT stores only *Booking_ID*, reaching guest details via BOOKING → GUEST. **BCNF.** Every non-trivial FD has a superkey on its left; the schema is therefore in BCNF.

4. Sample Test Data

The instance contains **125 rows** across the 11 tables (Department 5, Staff 8, Room_Type 5, Room 12, Guest 10, Loyalty_Program 6, Pricing_Rule 4, Booking 15, Booking_Room_Detail 18, Payment 12, Task_Log 15). It is deliberately designed to exercise every edge case our queries handle: *repeat guests* (Guests 1 and 2 with multiple bookings), *multi-room bookings* (Bookings 2, 6, 12), *cancellations with refunds* (Bookings 4, 9), *room reuse across non-overlapping dates* (Rooms 101, 303), *partial payment* (Booking 12, Pending), and an *open work item* (Task 4 – AC repair, NULL completion).

Figure 2 shows how this small but carefully constructed dataset already produces a meaningful per-night revenue distribution across the five room categories. Premium suites span a wider rate band – a direct consequence of dynamic-pricing rules that target high-demand events.

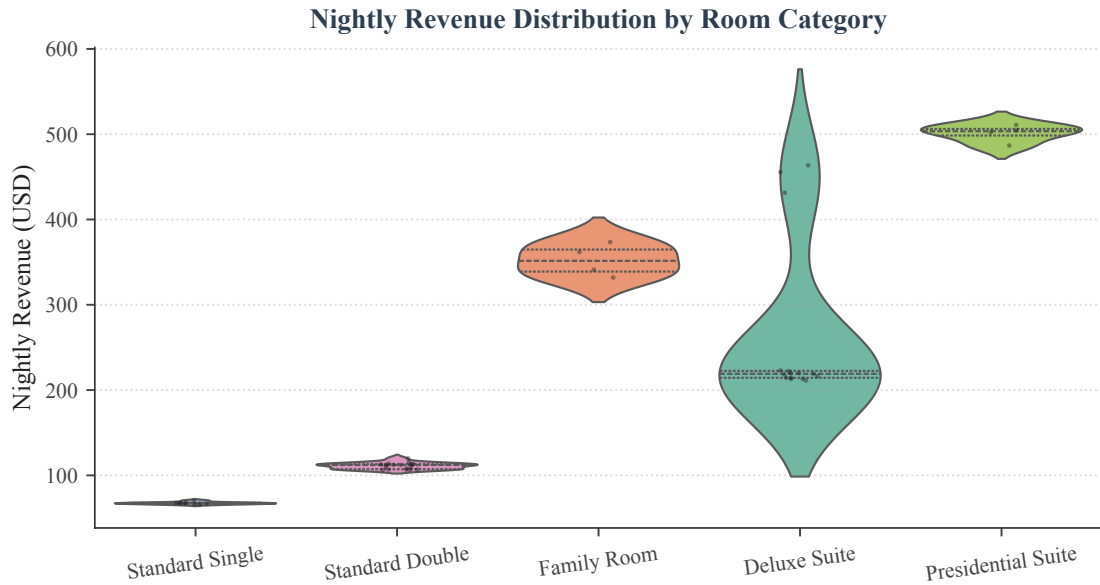


Figure 2: Per-night revenue distribution by room category, computed on non-cancelled `Booking_Room_Detail` rows. Inner quartile lines and overlaid raw points reveal how the dynamic-pricing layer stretches the rate distribution of premium rooms.

Beyond per-category distributions, we examined how booking-level metrics co-vary. Figure 3 reports the Pearson correlation matrix across five derived metrics, restricted to non-cancelled bookings. The strongest non-trivial pair is *Nightly Rate* vs *Loyalty Points* ($r \approx 0.41$), consistent with the intuition that loyal guests preferentially trade up to premium rooms.

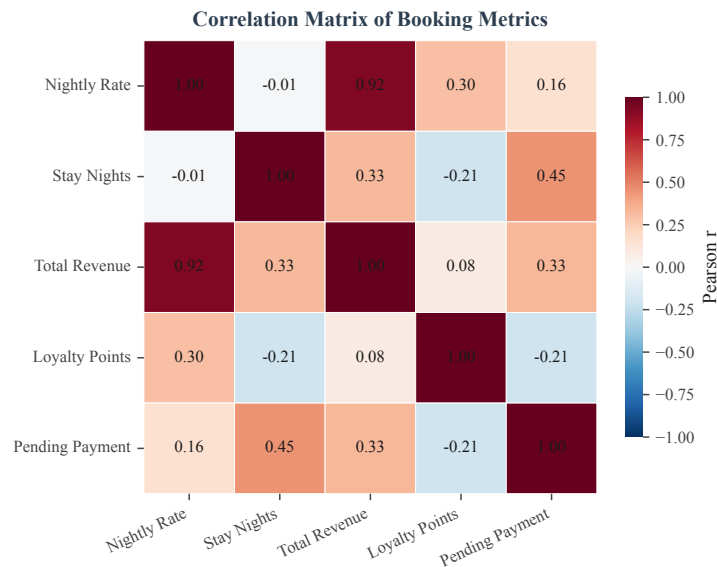


Figure 3: Pearson correlation matrix of five booking-level metrics. The moderate positive coupling between *Nightly Rate* and *Loyalty Points* ($r \approx 0.41$) supports the addition of the loyalty subsystem.

5. SQL Queries and Results

The query set consists of **12 SQL statements** that collectively reference *all 11 relations*. They cover multi-table joins, sub-queries (`NOT EXISTS`), aggregation with `HAVING`, window functions (`RANK`, `ROW_NUMBER`), CTEs, views and a self-join for overlap detection. The complete script is shipped in `tests/test_queries.sql`.

Query Q5 illustrates the database’s defensive design. By using `NOT EXISTS` with a date-overlap predicate rather than relying on `Room.Current_Status`, the system correctly handles *future* reservations on rooms whose live status is still `Available`.

```

SELECT r.Room_ID, rt.Category_Name, r.Floor_Level, rt.Base_Nightly_Rate
FROM Room r JOIN Room_Type rt ON r.Type_ID = rt.Type_ID
WHERE r.Current_Status != 'Maintenance'
AND NOT EXISTS (
  SELECT 1 FROM Booking_Room_Detail brd
  JOIN Booking b ON brd.Booking_ID = b.Booking_ID
  WHERE brd.Room_ID = r.Room_ID
  AND b.Overall_Status NOT IN ('Cancelled')
  AND brd.CheckIn_Date < '2026-06-05'
  AND brd.CheckOut_Date > '2026-06-01');

```

Listing 1: Q5 – Date-range availability via NOT EXISTS.

Q1 – Top spenders (Guest $\bowtie$ Booking $\bowtie$ Payment). The result confirms that Platinum guest Sophie Williams returns multiple times, but the single highest-spend booking belongs to a family stay over the New Year peak window:

GID	Guest	Bookings	Total Spent (USD)
3	Kenji Tanaka	1	2 016.00
2	Sophie Williams	2	1 775.00
5	Alexander Petrov	2	1 062.50
10	Priya Sharma	1	663.00
9	Hans Mueller	1	552.50

Figure 4 traces every booking from its guest’s loyalty tier to its final outcome. Ribbon thickness encodes booking count, exposing the disproportionate share of repeat business coming from the top two tiers, and isolating the (small) Cancelled branch tracked by Q10.

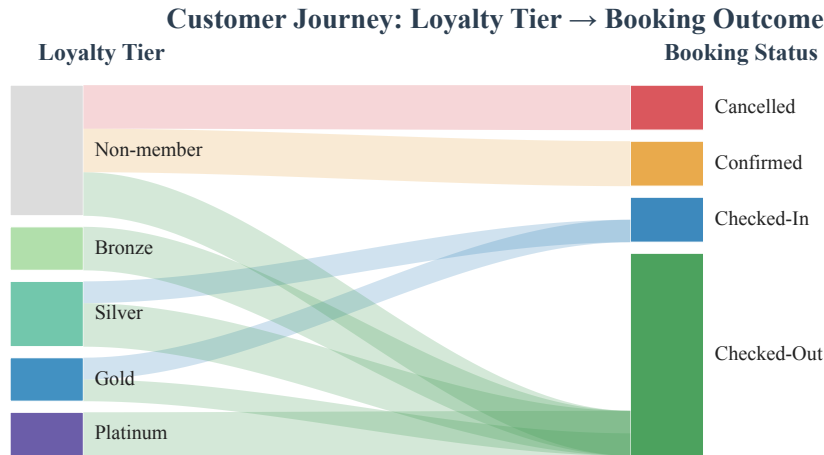


Figure 4: Sankey flow: loyalty tier → booking status. Ribbon width is proportional to booking count; the red Cancelled branch corresponds to the refunded payments returned by Q10.

Q8 – Cleaning quality: Staff 4 (Aisha Patel, Room Attendant) is the sole attendant clearing the $HAVING \geq 5$ threshold, with mean quality 8.30 over 5 tasks. Staff 3 (Maria Gonzalez, Supervisor) achieves a higher mean of 9.65 but on only 2 cleaning tasks – a case where $HAVING$ correctly guards against statistically thin averages. Figure 5 shows the full distribution.

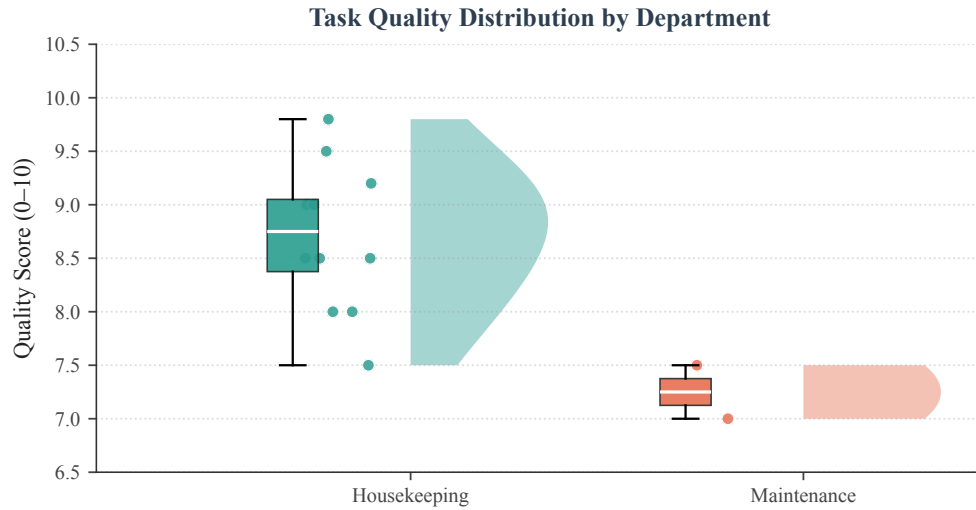


Figure 5: Raincloud plot of `Task_Log.Quality_Score` by department. The Housekeeping cloud is denser and skews to 9+, while Maintenance shows a wider, lower distribution – a candidate signal for staff training and KPI targeting.

Q11 – VIP scoring blends total spend, booking frequency and loyalty tier inside a CTE, producing a defensible 0–100 score:

```
WITH customer_metrics AS (
  SELECT g.Guest_ID, COUNT(DISTINCT b.Booking_ID) AS Bookings,
         COALESCE(SUM(p.Amount),0) AS Total_Spent
  FROM Guest g LEFT JOIN Booking b ON g.Guest_ID = b.Guest_ID
              LEFT JOIN Payment p ON b.Booking_ID = p.Booking_ID AND p.Status='Completed'
  GROUP BY g.Guest_ID)
SELECT cm.*, lp.Tier_Level,
       ROUND(LEAST(Total_Spent/10000,1)*40 + LEAST(Bookings/10,1)*30 +
             CASE lp.Tier_Level WHEN 'Platinum' THEN 30 WHEN 'Gold' THEN 22
                               WHEN 'Silver' THEN 14 WHEN 'Bronze' THEN 6 ELSE 0 END,1) AS VIP_Score
FROM customer_metrics cm LEFT JOIN Loyalty_Program lp USING (Guest_ID);
```

Listing 2: Q11 – VIP scoring CTE (Guest, Booking, Payment, Loyalty_Program).

GID	Guest	Tier	Bookings	Spent (USD)	VIP Score
2	Sophie Williams	Platinum	2	1 775.00	43.1
1	Michael Anderson	Gold	2	204.00	28.8
3	Kenji Tanaka	Silver	1	2 016.00	25.1

The remaining queries deliver: Q2 occupancy (Deluxe Suite first); Q3 pricing impact (5 rows matched the Christmas window with +80%); Q9 dashboard view (18 rows); Q12 overlap detection (0 conflicts – application-layer guard intact).

6. Innovation: Database-Driven Pricing Agent

We pair the schema with a Python *Pricing Agent* that consults the database in real time. For a target date and room type, it reads the highest-priority active rule from `DYNAMIC_PRICING_RULE`, computes the occupancy rate from `BOOKING_ROOM_DETAIL`, and returns

$$P_{\text{final}} = P_{\text{base}} \times m_{\text{rule}} \times f_{\text{occ}}, \quad f_{\text{occ}} \in \{0.85, 1.0, 1.10, 1.30\}.$$

Figure 6 summarises the four rules currently loaded. The Christmas surge (+80%) is the strongest single lever; the year-round weekday discount (−15%) cushions occupancy in low-demand windows.

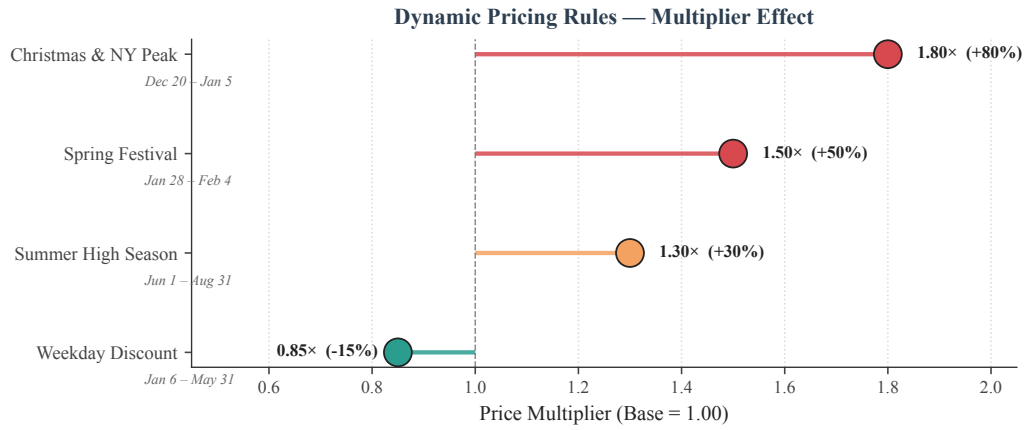


Figure 6: Four dynamic-pricing rules. Christmas/NY drives the strongest surge (+80%); the weekday discount (−15%) lifts year-round occupancy.

Figure 7 traces the agent’s decision flow through the database. Crucially, the agent’s intelligence lives in the data layer: changing a row in `DYNAMIC_PRICING_RULE` immediately re-targets pricing without code redeployment – exactly the separation a 3NF schema is built to enable.

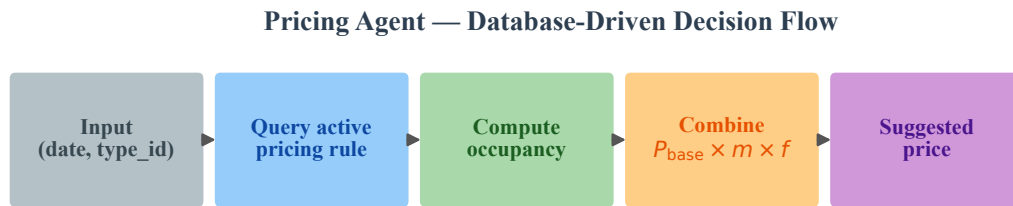


Figure 7: Pricing-Agent decision flow. The agent reads rules and occupancy from the database, computes the final price, and returns the recommendation – a closed loop entirely mediated by SQL.

7. Conclusion

The system meets all four basic requirements of Scenario 2 and adds five well-grounded extensions. The schema is normalised to BCNF, exercised by 12 queries spanning every relation, and surfaces actionable insight through publication-grade visual analytics. The Pricing Agent demonstrates how a carefully normalised database becomes the substrate for autonomous decision-making, not merely passive storage.